Simple Pendulum

Faizur Rahman
Department of Chemical Engineering | BIT Sindri
Registration No. 23030420034

1 Introduction

A simple pendulum is an ideal case of a swinging mass: we take a point mass m hanging from a light, inextensible string (or thin rod) of length L (Figure 1). The angle θ is measured from the downward vertical, so θ = 0 is the resting position. If the bob is displaced and released, it swings back and forth about this position.

Two forces act on the bob — its weight mg and the tension T in the string. It helps to resolve these along the string (radial, $\hat{r}$) and along the arc of motion (tangential, $\hat{\theta}$). Because the string length is fixed, the radial part of gravity is balanced by tension ($F_r = -T$) and does not change L. Only the tangential component drives the motion:

$$F_\theta = -mg \sin\theta \qquad (1)$$

Applying Newton's second law along the path, with tangential acceleration $a = L\ddot{\theta}$, gives

$$a = L\ddot{\theta}\hat{\theta} \qquad (2)$$

and therefore the governing equation

$$L\ddot{\theta} = -g \sin\theta \qquad (3)$$

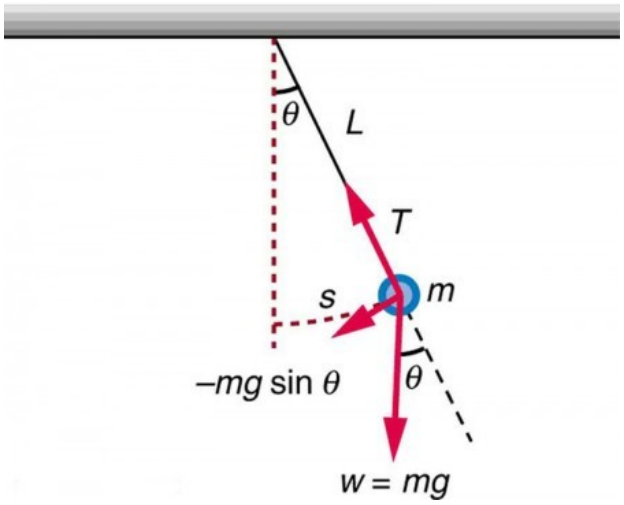

Figure 1: Simple pendulum of mass m at angle θ from the vertical, showing tension T, weight mg, and the restoring component −mg sin θ.

2 Analytical Approximation

Eq. (3) is nonlinear because of sin θ, so a closed-form solution is not elementary. When the swing is small ($\theta \ll 1$ radian), $\sin\theta \approx \theta$ and the equation simplifies to

$$L\ddot{\theta} \approx -g\theta \qquad (4)$$

with solution (released from rest at $\theta = \theta_0$)

$$\theta = \theta_0 \cos(\omega t) \qquad (5)$$

Here $\omega = \sqrt{(g/L)}$ is the angular frequency. The motion is simple harmonic and the period is $T = 2\pi/\omega = 2\pi\sqrt{(L/g)}$, which does not depend on the amplitude in this limit.

3 Numerical Solution

For larger angles we integrate Eq. (3) on the computer. First rewrite it as two first-order equations by setting $\omega = \dot{\theta}$ (here ω is angular velocity, not the frequency above):

$$d\omega = -(g/L) \sin\theta \, dt \qquad (6)$$

$$d\theta = \omega\, dt \qquad (7)$$

Differentials are replaced by small steps Δt (forward Euler). Starting from $\theta(0) = \theta_0$ and $\omega(0) = 0$, we update θ and ω at each step. After n steps the time is $t = n\,\Delta t$. A smaller Δt gives a more accurate trajectory.

4 Sample Problem

Take a 40 kg steel bob (radius about 10 cm) on a 25 m wire — a large laboratory-scale pendulum. For small swings the expected period is $T = 2\pi\sqrt{(L/g)} \approx 10$ s (with $g = 9.81$ m/s² we get $T = 10.030$ s). In the simulation below we release it from rest at two starting angles, 15° and 60°, and compare with the small-angle formula.

5 Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt

plt.style.use('bmh')
figsize = (12, 5)
dpi = 150  # resolution in dots per inch
g = 9.81   # m/s^2
L = 25     # m
m = 40     # kg

def analytical_approximation(t, theta0):
    return theta0 * np.cos(t * (g / L) ** 0.5)

def RHS(theta, w, dt):
    """Return the right-hand side of the ODE describing a simple pendulum."""
    dw = -np.sin(theta) * dt * g / L
    dtheta = w * dt
    return dw, dtheta

def integrate_one_step(theta, w, dt):
    """Performs one step of integration."""
    dw, dtheta = RHS(theta, w, dt)
    w = w + dw
    theta = theta + dtheta
    return w, theta

def integrate_n_steps(theta0, w0, dt, n):
    """Performs integration for n time steps."""
    theta = (n + 1) * [0]
    w = (n + 1) * [0]
    theta[0] = theta0
    w[0] = w0
    for i in range(n):
        w[i + 1], theta[i + 1] = integrate_one_step(theta[i], w[i], dt)
    return w, theta
```

Next we integrate for $\theta_0 = 15°$ and 60° over 20 s (n = 10000 steps) and overlay the analytical approximation on the same plot:

```
theta0_1 = np.pi / 12   # 15 degrees
theta0_2 = np.pi / 3    # 60 degrees
T = 20
n = 10000
t = np.linspace(0, T, n + 1)
dt = T / float(n)

w1, theta1 = integrate_n_steps(theta0_1, 0, dt, n)
w2, theta2 = integrate_n_steps(theta0_2, 0, dt, n)

plt.figure(figsize=figsize, dpi=dpi)
plt.title("Angular position")
plt.plot(t, theta1, "m",
         label=r"$\theta_0=%.0f^\circ$" % (theta0_1 * 180 / np.pi))
plt.plot(t, analytical_approximation(t, theta0_1), "m--",
         label="Approximation")
plt.plot(t, theta2, "g",
         label=r"$\theta_0=%.0f^\circ$" % (theta0_2 * 180 / np.pi))
plt.plot(t, analytical_approximation(t, theta0_2), "g--",
         label="Approximation")
plt.xlabel(r"$t$, [s]")
plt.ylabel(r"$\theta(t)$, [rad]")
plt.legend()
plt.show()
```

For 15° the numerical curve stays close to the cosine approximation. At 60° the two curves separate clearly — the small-angle assumption no longer holds. A better ODE solver would reduce numerical error further, but the main mismatch here is physical, not just the integrator.

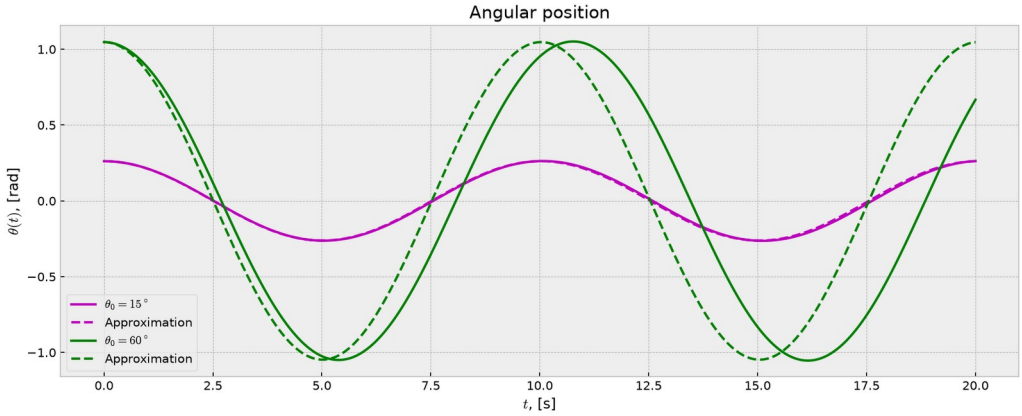

Figure 2: Angular position for θ_0 = 15° and 60° (solid) vs small-angle approximation (dashed).

6 Conservation of Energy

If there is no friction, total mechanical energy

$$E = U + K = mgL(1 - \cos\theta) + \tfrac{1}{2} m L^2 \theta^2 \qquad (8)$$

should stay nearly constant. Plotting U, K and E for the 60° run is a quick check that Δt is small enough:

```
def compute_PE(theta):
    return m * g * L * (1 - np.cos(theta))  # potential energy

def compute_KE(w):
    return 0.5 * m * L**2 * np.asarray(w)**2  # kinetic energy

plt.figure(figsize=figsize, dpi=dpi)
plt.title(r"Mechanical energy, $\theta_0=%.0f^\circ$"
          % (theta0_2 * 180 / np.pi))
plt.plot(t, compute_PE(theta2), label="Potential energy")
plt.plot(t, compute_KE(w2), label="Kinetic energy")
plt.plot(t, compute_PE(theta2) + compute_KE(w2), label="Total energy")
plt.xlabel(r"$t$, [s]")
plt.ylabel(r"$E$, [J]")
plt.legend(loc=1)
plt.show()

def compute_error(w, theta):
    """Computes the relative error."""
    E0 = compute_PE(theta[0]) + compute_KE(w[0])
    E1 = compute_PE(theta[-1]) + compute_KE(w[-1])
    return np.abs((E0 - E1) / E0)

print("Relative change in E:")
print("Theta = %.0f: %.2e" % (theta0_1 * 180 / np.pi, compute_error(w1, theta1)))
print("Theta = %.0f: %.2e" % (theta0_2 * 180 / np.pi, compute_error(w2, theta2)))
```

Total energy barely drifts, so the chosen step size is acceptable. The relative change $|E_0 - E_1|/E_0$ for the two runs is:

Relative change in E:

θ = 15° : 1.56×10^{-2}

θ = 60° : 1.33×10^{-2}

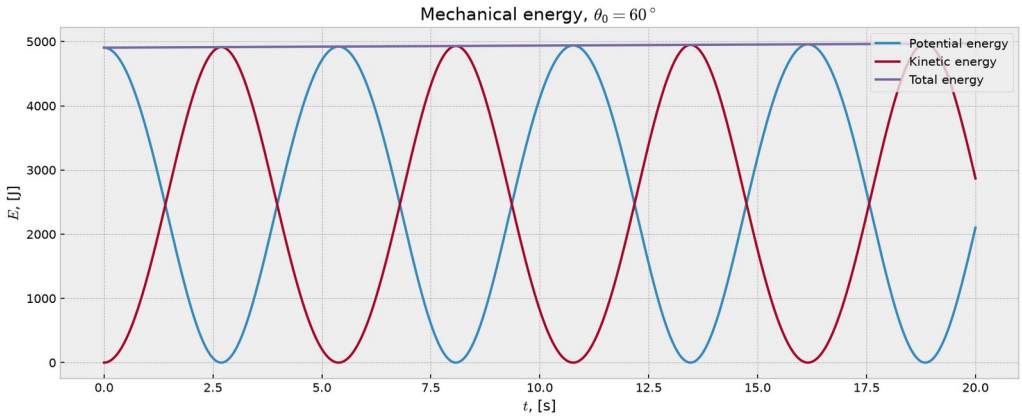

Figure 3: Potential, kinetic and total energy for θ_0 = 60°.